

# Lab 5 Build Guide

## Lab 5 Build Guide: Post-Change Validation and Testing

This guide walks through the test framework that validates the network after every deployment. By the end you will have a pytest suite that checks three things: is the config on the device what we intended, are the protocols actually working, and is the device healthy. This is the difference between “I pushed the config” and “the network is operating correctly.”

### What We Built

Lab 4 gave us a deployment pipeline. Configs go from Git to the routers. But pushing a config and having a working network are two different things. A BGP neighbor might be configured but stuck in Connect because the underlay is broken. An interface might have the right IP but be physically down. A daemon might have crashed after the config was loaded. Lab 5 catches all of these.

The test framework has three categories that map to three questions:

Configuration verification asks “is it configured correctly?” It compares the running config on each device against what the data model says should be there. If OSPF should have router-id 10.0.0.1, it checks that the running config contains that line.

Operational state testing asks “is it working correctly?” It checks that OSPF adjacencies are in Full state, BGP sessions are Established, loopback routes are present, and devices can ping each other. These tests would catch a config that loaded cleanly but produced a non-functional network.

Health checks ask “is it operating within parameters?” They verify that the FRR daemons are running, interfaces are up without errors, the routing table has a sane number of routes, and vtysh is responsive. These tests catch silent failures that do not affect protocol state immediately but indicate a problem building.

### Prerequisites

You need Labs 1-4 complete with the ContainerLab topology running and configs deployed. Verify with:

```
uv run python -m scripts.fabric_status
```

All 6 devices should show UP with OSPF and BGP neighbors. If not, deploy first:

```
uv run python -m generators.python.render
uv run python -m deploy.scrapli_deploy
```

Wait 30 seconds for convergence, then check again.

## Part 1: Test Fixtures

The test fixtures live in `tests/post_change/conftest.py`. They provide two things: access to the running devices and access to the validated data model.

```
cat tests/post_change/conftest.py
```

### DeviceConnection

The `DeviceConnection` dataclass wraps `docker exec` calls to a `ContainerLab` container. It has two methods: `exec()` for running arbitrary shell commands and `vttysh()` for running FRR CLI commands. Every test that needs to talk to a device gets a `DeviceConnection` through the `fabric_devices` or `device_map` fixture.

### Session Scope

All fixtures use `scope="session"` which means the data model is loaded once and reused across all tests. This is important because loading 8 YAML files and validating them through Pydantic on every test would add seconds of overhead for no benefit.

### Data Model Fixtures

The `topology`, `underlay`, `overlay`, and `fabric` fixtures provide direct access to the validated Pydantic model instances. Tests use these to know what the expected state should be. The data model is the single source of truth for what “correct” looks like.

## Part 2: Configuration Verification Tests

```
cat tests/post_change/test_config_verify.py
```

These tests pull the running config from each device and check for expected content. They do not parse the FRR config into structured data. Instead, they use string matching because the running config is the authoritative representation of what FRR is actually doing.

### What Is Tested

`TestHostnames` verifies that each device’s hostname matches the topology. This sounds trivial but catches the case where a config was pushed to the wrong device.

TestInterfaceAddressing checks that every loopback and fabric link IP address from the data model appears in the running config. It checks both sides of every P2P link.

TestOSPFFConfig verifies the OSPF router-id, OSPF area assignments on interfaces, and point-to-point link type. These are the three things that must be correct for OSPF adjacencies to form.

TestBGPConfig checks the ASN, router-id, cluster-id on route reflectors, route-reflector-client designations, and total neighbor count. The neighbor count test uses the data model to compute the expected number: RRs should have (total devices - 1) neighbors, non-RRs should have (number of RRs) neighbors.

## Run It

```
uv run pytest tests/post_change/test_config_verify.py -v
```

You should see 11 tests pass.

## Part 3: Operational State Tests

```
cat tests/post_change/test_operational.py
```

This is the layer that catches “configured but not working” failures. The config tests can all pass while the network is broken.

## What Is Tested

TestOSPFSState checks that every device has the correct number of OSPF neighbors and that all of them are in Full state. The expected count comes from the link data: if a device has 4 fabric links, it should have 4 OSPF neighbors.

TestBGPState verifies that all BGP sessions are Established (not Connect or Active), the EVPN address family is active, and the peer count matches expectations.

TestRoutePresence checks that every loopback appears as an OSPF route on every other device, and that each device has at least one OSPF-learned route. If spine1 cannot see leaf2’s loopback in its routing table, something is wrong with the underlay.

TestPingMesh does actual ping tests between device types: spine to leaf, spine to border, and leaf to leaf. These tests prove end-to-end data plane reachability, not just control plane state. A routing table might show a route, but the packets might not actually get through.

## Run It

```
uv run pytest tests/post_change/test_operational.py -v
```

You should see 10 tests pass.

## Part 4: Health Check Tests

```
cat tests/post_change/test_health.py
```

Health checks catch problems that are not yet affecting protocol state but indicate something is wrong or about to go wrong.

### What Is Tested

TestDaemonHealth verifies that zebra, ospfd, bgpd, and watchfrr are all running as processes inside each container. If ospfd crashes after deployment, OSPF state will eventually time out, but the health check catches it immediately.

TestInterfaceHealth confirms that all fabric-facing interfaces are in UP state at the Linux kernel level. An interface might be configured in FRR but down at the OS level.

TestRouteTableHealth checks that each device has at least 6 routes (a sanity floor) and that OSPF is not injecting a default route, which would be a design violation in this fabric.

TestFRRResponsiveness runs vtysh commands and checks that FRR responds with valid output. If vtysh hangs or returns empty output, the device is in trouble even if the daemons show as running.

### Run It

```
uv run pytest tests/post_change/test_health.py -v
```

You should see 10 tests pass.

## Part 5: Running the Full Suite

Run all three test categories together:

```
uv run pytest tests/post_change/ -v
```

31 tests across three categories. To generate an HTML report:

```
uv run pytest tests/post_change/ -v --html=reports/post-change-report.html --self-contained-html
```

The report shows each test with its result, timing, and any failure details. This is the artifact you would attach to a deployment ticket or PR to prove the change was validated.

### Integration with the Pipeline

In Lab 4's deploy workflow, you would add a post-deployment test step:

```
- name: Run post-change validation
  run: uv run pytest tests/post_change/ -v --tb=short
```

This means every merge to main triggers: validate the data model, generate configs, deploy to devices, then verify the deployment worked. If any post-change test fails, the workflow fails and the team is notified.

## Part 6: Break Something on Purpose

The real value of these tests shows when something is wrong. Here are demos for each test category.

### Config Verification Failure

Connect to spine1 and remove the cluster-id manually:

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "conf t" -c
    "router bgp 65000" -c "no bgp cluster-id" -c "end"
```

Now run the config verification tests:

```
uv run pytest tests/post_change/
    test_config_verify.py::TestBGPCfg::test_rr_cluster_id
    -v
```

It fails because the running config no longer matches the intent. Redeploy to fix:

```
uv run python -m deploy.scrapli_deploy
```

### Operational State Failure

If you had a device with a broken OSPF adjacency (which you can simulate by removing the IP from an interface), the operational tests would catch it even though the config tests pass. The config says the IP should be there, but the OSPF neighbor is not in Full state.

### Health Check Failure

If you stopped the ospfd process on a device, the health check would catch it immediately, before the OSPF adjacency times out (which takes 40 seconds with the current dead interval).

## Part 7: Commit the Test Framework

```
git status
```

You should see the new tests/post\_change/ directory with 4 files.

```
git add tests/post_change/ docs/lab5-build-guide.md
git commit -m "Lab 5: post-change validation with config,
              operational, and health tests"
```

Check the log:

```
git log --oneline
```

Five commits. Data model, validation framework, config generation, CI/CD pipeline, and now post-change testing. The automation stack now covers the full lifecycle: validate the intent, generate the config, deploy it, and prove it worked.

## Part 8: What We Proved

By the end of this lab you have demonstrated three things.

First, that “deployed” and “working” are different assertions. Configuration verification confirms the right commands are on the device. Operational state testing confirms the protocols formed adjacencies and sessions. Health checks confirm the device is operating within normal parameters. Each layer catches a different class of failure.

Second, that tests are driven by the data model. The expected OSPF neighbor count is not hardcoded. It is computed from the link data. The expected BGP peer count is derived from the route reflector configuration. If you add a new device to the data model, the tests automatically expect the new adjacencies and sessions without any test code changes.

Third, that post-change testing closes the automation loop. The pipeline now validates before deployment (Lab 2), generates configs (Lab 3), deploys them (Lab 4), and verifies the result (Lab 5). Every step is automated, every step produces evidence, and every step blocks the pipeline if it fails.

## Troubleshooting

**Tests fail with “container not found”:** The ContainerLab topology is not running. Deploy it first with `sudo containerlab deploy --topo containerlab/topology.yaml`.

**OSPF neighbor count is wrong:** The test expects the number of neighbors to match the number of fabric links per device. If you added or removed links in the data model without redeploying, the test will fail. Redeploy and wait 30 seconds for convergence.

**BGP peers in Connect state:** OSPF has not converged yet. Wait 30 seconds after deployment. If it persists, check that the loopback has `ip ospf area` configured (the fix from Lab 4).

**Ping tests fail but OSPF/BGP pass:** This can happen if the data plane is not forwarding even though the control plane is up. In ContainerLab this is rare but check `ip -br link` to verify interfaces are UP at the kernel level.

**Health tests report daemon not running:** The daemon check uses `ps aux` to find FRR processes. If the container was recently restarted, daemons may take a few seconds to start. Re-run the tests after a brief wait.

**Tests are slow:** The suite runs about 18 seconds because it makes docker exec calls to 6 containers. This is expected. The session-scoped fixtures minimize overhead by loading the data model once.